

H0002 — Definir categorías

Sistema de Gestión de Torneos Deportivos de Contacto (Taekwondo)

- Épica: Configuración inicial
- 5 puntos
- KANO: Básico
- Rol: Coordinador (alta)

1. Qué se implementó

La historia permite al **Coordinador** definir las categorías de competencia de un torneo (combate/formas, sexo, y rangos de edad, peso y graduación) y a **todos los roles** consultarlas. Se cubrió el flujo completo: **alta** y **listado** de categorías por torneo.

Decisión de alcance: la entidad `Categoria`, su configuración EF y la tabla `categorias` ya existían desde la migración `InitialCreate`; el trabajo se concentró en las capas Application, API y el frontend (más un ajuste de modelo en la v2, ver §7).

2. Backend — capas (Clean Architecture + CQRS)

Capa	Archivos clave
Application	<code>Features/Categorias/</code> — <code>CategoriaResponse</code> , <code>CreateCategoriaCommand/Handler/Validator</code> , <code>GetCategoriasByTorneoQuery/Handler</code> .
Infrastructure	<code>CategoriaRepository</code> (EF Core) + registro en <code>DependencyInjection</code> .
API (FastEndpoints)	<code>Endpoints/Categorias/</code> — <code>CreateCategoriaEndpoint</code> (POST, rol Coordinador) y <code>GetCategoriasByTorneoEndpoint</code> (GET, todos), con sus Validators.

Método	Ruta	Rol
GET	<code>/api/v1/torneos/{torneoId}/categorias</code>	Todos
POST	<code>/api/v1/torneos/{torneoId}/categorias</code>	Coordinador

Flujo de alta

```
POST /torneos/{id}/categorias
-> CreateCategoriaRequestValidator (FastEndpoints valida el request)
-> CreateCategoriaEndpoint (arma el Command y lo envia con MediatR)
-> ValidationBehavior (pipeline: valida el Command)
-> CreateCategoriaCommandHandler (verifica torneo, convierte enums, persiste)
-> CategoriaRepository.AddAsync (EF Core -> PostgreSQL)
-> 201 Created + CategoriaResponse
```

3. Decisiones de diseño base

Enums como texto en los bordes. El dominio usa enums fuertes (`TipoCompetencia` , `Sexo` , `Graduacion`), pero la API los expone/recibe como texto ("Combate", "F"), validados con `Enum.TryParse` . Consistente con `TorneoResponse.Estado` .

Existencia del torneo. Alta y listado verifican el torneo y lanzan `NotFoundException` (→ 404) — sin categorías huérfanas.

4. Frontend — React 19 + TanStack Query

- `types/index.ts` — `Categoria` , `CreateCategoriaInput` , lista `GRADUACIONES` .
- `lib/api.ts` — `categorias.listar/crear` .
- `hooks/useCategorias.ts` — query keys por torneo; mutación invalida la lista en `onSuccess` .
- `components/categorias/` — `CategoriaForm` (RHF + Zod), `CategoriaList` , `CategoriaCard` .
- `routes/torneos/$torneold/categorias.tsx` — lista para todos + form solo Coordinador.

Cumple WCAG 2.2 AA: labels visibles, targets ≥44px, foco visible, HTML semántico, errores accionables.

5. Pruebas y verificación

- ✓ Backend compila con 0 errores · **31/31 tests** en verde.
- ✓ Frontend `tsc + vite build` sin errores.

6. Corrección transversal — fetch wrapper v2

Bug detectado en runtime: el listado mostraba "One or more errors occurred" .
Causa: `apiFetch` enviaba `Content-Type: application/json` en *todas* las peticiones; en un GET sin body, `FastEndpoints` intentaba deserializar un JSON vacío y devolvía 400.
Fix: (1) enviar `Content-Type` solo cuando hay body; (2) extraer el mensaje real de `errors` (objeto `{campo:[msg]}` en `FastEndpoints`), no del `message` genérico. Documentado en `frontend/CLAUDE.md` (reglas 9-10) y `backend/CLAUDE.md` .

7. Ajustes de negocio / UX v2

Sobre la versión inicial se aplicaron seis mejoras solicitadas:

#	Mejora	Implementación
1	Color de la pill por tipo	Formas azul Combate ámbar — el rojo se mantiene reservado a los CTAs (regla 60-30-10).
2	Formas sin peso	El form oculta los campos de peso en Formas; el handler descarta el peso (guarda <code>null</code>) y la validación de peso solo corre en Combate.
3a	Graduación igual → una sola	Si mín == máx, la card muestra "Cinturón Blanco" en lugar de "...Blanco a ...Blanco".

3b	Rangos de edad abiertos	"Hasta 6 años" (sin mínimo) y "35 años en adelante" (sin máximo). Se logra dejando el campo vacío; la card lo formatea.
4	Edad/peso no pueden ser iguales	Validación <code>mín < máx</code> estricta (front y back) cuando ambos extremos existen. Se usan dos inputs "Desde/Hasta" (descartado el slider por precisión y accesibilidad).

Cómo se resolvió el conflicto entre el punto 3b y el 4: los extremos abiertos se modelan como `null` (no como valores iguales). Así "Hasta 6" = `{min:null, max:6}` y "+35" = `{min:35, max:null}`, y la regla `mín < máx` solo aplica cuando ambos existen — ambas reglas conviven sin contradicción. La **graduación sí admite mín == máx** porque ahí un único cinturón es un valor legítimo, no un rango vacío.

Cambio de modelo (rediseño de `Categoria`)

- Entidad + DTOs: `RangoEdadMin/Max ( int? )` y `RangoPesoMin/Max ( decimal? )` ahora **nullable**.
- Config EF: se agregó `HasPrecision(5,2)` al peso.
- Migración `CategoriaRangosOpcionales` (solo `AlterColumn` a nullable — sin pérdida de datos). Se auto-aplica al arrancar la API (`MigrateAsync`).
- Tests: +7 (edad/peso iguales fallan, rangos abiertos válidos, Formas descarta peso, graduación igual válida).

8. Checklist final

Ítem	Estado
Alta + listado (backend, CQRS, FastEndpoints, roles)	✓
Repositorio + DI + Mapster	✓
Frontend (types, api, hook, form, card, ruta) responsive + WCAG 2.2 AA	✓
Fix del fetch wrapper (Content-Type / parseo de errores)	✓ v2
Pill ámbar/azul · Formas sin peso · rangos abiertos · mín<máx · graduación única	✓ v2
Modelo nullable + migración	✓ v2 (auto-aplica al arrancar)
Comentarios XML <code><summary></code> en todo el código nuevo	✓
Backend 31/31 tests · frontend build OK	✓